

GG_registro_Pro

Sistema experto local basado en agentes inteligentes para recomendación y gestión de lectura

Proyecto académico desarrollado para demostrar ingeniería del conocimiento, motores de inferencia, explicabilidad, base de datos, agentes inteligentes e ■■■■■ acción natural con IA sobre libros.

Repositorio: Gabbisita / VibeandBloom_KenyaGabrielaFrutosGonzalez

Fecha: 12 de junio de 2026

1. Objetivo

Construir un sistema experto moderno que converse naturalmente sobre libros, detecte intención, infiera preferencias lectoras, genere recomendaciones actuales y explique las decisiones tomadas por el sistema.

2. Descripción del sistema

Novelia es una plataforma local cliente-servidor orientada a la lectura. El usuario puede registrarse, iniciar sesión, consultar recomendaciones modernas tipo BookTok/Bookstagram, hablar con un chatbot lector, guardar libros en su biblioteca, registrar progreso, escribir reseñas y consultar estadísticas de lectura.

El sistema integra IA generativa cuando existe conexión a Gemini, pero mantiene un modo local de respaldo para no depender exclusivamente de internet y conservar funcionalidad básica.

3. Arquitectura

Arquitectura de tres capas: frontend en React, backend en FastAPI y persistencia con SQLite. Los agentes se ejecutan en el backend y se coordinan por rutas HTTP.

Agentes principales: atención al cliente y detección de vibe, generador de pedido/recomendación con reglas, y supervisor-explicador que resume, justifica e interpreta el razonamiento.

4. Base de conocimiento

La base de conocimiento se compone de: perfiles de usuario, biblioteca, reseñas, sesiones de vibe y actividades lectoras. A partir de esos datos se calculan hábitos, racha, nivel lector, desafíos, progreso semanal y preferencias de estilo.

Reglas destacadas: si el usuario tiene más de 20 libros leídos, se clasifica como lector experto; si el promedio de reseñas es alto, se priorizan obras mejor valoradas; si el progreso llega a 100%, el libro pasa automáticamente a leído.

5. Inferencias utilizadas

- IF vibe = romantico THEN recomendar romance, contemporary romance y títulos populares en BookTok.
- IF vibe = misterioso THEN priorizar thriller, mystery y suspense.
- IF libros_leidos > 20 THEN priorizar libros menos conocidos para ampliar el horizonte lector.
- IF calificacion_promedio >= 4.5 THEN sugerir libros con mejor recepción.
- IF progreso >= 80% THEN sugerir libros similares.
- IF progreso = 100% THEN marcar como leído y sugerir reseña.

6. Explicación de agentes

Agente 1 - Atención al Cliente: analiza texto, canción, imagen o video para detectar vibe, intención y señales emocionales. Responde de forma natural y orienta la búsqueda de libros.

Agente 2 - Generador de Pedido: interpreta la información del usuario, aplica reglas de recomendación, valida la biblioteca y guarda libros o progreso en la base de datos.

Agente 3 - Supervisor / Explicador: resume recomendaciones, explica inferencias y genera un cierre comprensible para el usuario.

7. Base de datos utilizada

Se utiliza SQLite de forma local en el archivo novela.db. Las tablas principales son usuarios, libros, biblioteca, reseñas, sesiones_vibe y actividades_lectura.

Esta estructura permite persistir el conocimiento del sistema y reconstruir el perfil del lector con explicaciones auditables.

8. Herramientas utilizadas

- Python
- FastAPI
- React
- SQLite
- Ollama/Gemini opcional
- Requests
- SQLAlchemy
- ReportLab para este PDF

9. Manual de usuario

Instalación: 1) entrar al proyecto, 2) activar entorno virtual en backend, 3) instalar dependencias de backend y frontend, 4) crear archivo .env si se desea usar Gemini, 5) ejecutar backend y frontend en terminales separados.

Backend: desde backend ejecutar `uvicorn main:app --reload`. El backend crea las tablas automáticamente y expone login, registro, vibe, biblioteca, reseñas, perfil y chat.

Frontend: desde frontend ejecutar `npm install` y `npm start`. La interfaz se conecta al API local configurado en `frontend/src/config.js`.

Uso de inferencias: el usuario escribe una vibra, canción, imagen o video; el sistema detecta la emoción, recomienda libros, explica por qué y permite guardarlos en biblioteca.

Uso del chatbot: desde la pantalla principal se puede preguntar por recomendaciones, inferencias, racha, progreso o qué leer según el estado de ánimo.

Conexión a base de datos: por defecto se usa SQLite local. Para cambiar, definir `DATABASE_URL` en `backend/.env`.

10. Capturas del sistema

En la versión final del documento se recomienda insertar capturas de: pantalla principal, recomendación por vibe, chatbot lector, biblioteca, perfil con estadísticas y flujo de reseñas.

11. Video demostrativo

Enlace de YouTube: agregar aquí la URL final del video demostrativo una vez publicado.

12. Conclusiones

Novelia demuestra un sistema experto moderno, explicable y ejecutable de forma local, con agentes inteligentes, reglas claras y una experiencia de conversación natural sobre libros.

El diseño visual original se conserva; los cambios realizados se enfocaron en robustez, inferencia y funcionalidad real del proyecto.

Elemento	Estado
Backend local	Operativo
Frontend	Compila correctamente
Chatbot lector	Activo
Perfil con estadísticas	Activo
PDF académico	Generado